

Practical Problems: Nested Loops, Lists, and Recursive Code

Learning Outcome Addressed

- Recognize, interpret, and write programs with nested loops.
- Recognize and interpret basic recursive functions.
- Translate simple recursive algorithms to Python functions.

This assignment allows you to apply what you have learned in this module about nested loops and lists and recursive functions and algorithms. Follow the instructions below to complete problems one to seven. If you get stuck on a problem and are not making any progress, please reach out to the learning facilitator for assistance.

1. [15 pts] *get_alpha_string*

Write the function **get_alpha_string(L)** that takes a 2D list of strings and returns a string containing only the alphabetical letters (a-z and A-Z) that appeared in the list, reading left-to-right, top-to-bottom.

For example, `get_alpha_string([ ['0', '1', 'a'], ['2', 'c', 'e'], ['3', '4', '5'] ])` would return 'ace', because 'a', 'c', and 'e' are the only alphabetical letters that appear in the list, 'a' occurs in the top row, and 'c' and 'e' occur in the middle row (with 'c' appearing first).

2. [15 pts] *count_multiples_with_end_digit*

Write the function **count_multiples_with_end_digit(n, digit)** that takes two integers and computes all the multiples in the multiplication table from 1 to n, i.e.

$1 \cdot 1, 1 \cdot 2, 1 \cdot 3 \dots$ all the way to $n \cdot (n - 2), n \cdot (n - 1), n \cdot n$, and returns the number of multiples that end with the given digit.

For example, `count_multiples_with_end_digit(6, 2)` would check all 36 entries in the multiplication table from 1×1 to 6×6 and return 6 because six multiples end in 2 ($1 \cdot 2 = 2, 2 \cdot 1 = 2, 2 \cdot 6 = 12, 3 \cdot 4 = 12, 4 \cdot 3 = 12$, and $6 \cdot 2 = 12$).

Hint: to check all the possible multiples, use a nested loop!

3. [20 pts] *process_payroll*

Write a function **process_payroll** that takes a company payroll (2D list of strings and numbers) as input, computes the pay owed to each employee based on their hours worked and pay rate, and returns a 2D list containing payment information (strings and numbers).

The payroll is formatted so that every inner list represents a single employee. The first element in the inner list is the employee's name, the second is their pay rate per hour, and all remaining elements are the hours worked per day for that pay period. The returned payment information is also formatted so that every inner list represents a single employee, but this list only has two elements per inner list: the employee's name, and the amount they should be paid for that period.

For example, if we call the function on the payroll:

```
[ [ "Tom", 20.00, 10, 12, 7, 9, 11 ],  
  [ "Leslie", 18.50, 10, 10, 10, 10, 9 ],  
  [ "Tobias", 16.75, 6, 12, 6.5, 11, 6 ] ]
```

We would return:

```
[ [ "Tom", 980.0 ],  
  [ "Leslie", 906.5 ],  
  [ "Tobias", 695.125 ] ]
```

Because Tom worked for $10+12+7+9+11=49$ hours, and $49 \cdot 20.0 = 980.0$; Leslie worked for $10+10+10+10+9=49$ hours, and $49 \cdot 18.50 = 906.5$; and Tobias worked for $6+12+6.5+11+6=41.5$ hours, and $41.5 \cdot 16.75 = 695.125$.

4. [15 pts] recursive_match

Write the function **recursive_match** which takes two lists and recursively returns the number of indexes where the two lists both hold the same value. For example, `recursive_match([4, 2, 1, 6], [4, 3, 7, 6])` would return 2 because the two lists both have 4 in the 0th index and 6 in the 3rd index.

You must solve this problem recursively. Here's a high-level approach for how to solve the problem:

1. The base case should check whether either list is empty. If this is true, 0 should be returned (once one list is empty, it cannot possibly match elements to the other).
2. The recursive case should recursively call the function on all but the first element of the two lists and store the temporary result.
3. The recursive case should also check whether the first elements are equal. If they are, 1 should be added to the result. Then the result can be returned.

5. [15 pts] recursive_sum_digits

Write the function **recursive_sum_digits(n)** that takes in a possibly negative integer, `n`, and recursively returns the sum of all the digits.

6. [20 pts] recursive_mirror

Write the function **recursive_mirror(s)** which takes a string and returns a recursively-generated 'mirror' version of the string. This is just a version of the string that has itself forwards followed immediately by itself backwards. For example, `recursive_mirror("hello")` would return "helloolleh".

Note: you must use recursion to solve this problem, and you may not use a slice with a negative step.

Hint: consider what the recursive call will return if the function works correctly. How can you use the recursive result once you make this assumption?

7. [25 pts] generate_inputs

Every program you write eventually gets converted down to lower-level code that your machine's hardware can read and run on circuits. Circuits take inputs in the form of electricity (low or high voltage) and transform the inputs through a series of hardware gates to eventually create an output voltage (low or high). We typically represent these voltages with binary numbers - 0 for low voltage, 1 for high voltage.

Given a single circuit with n inputs, we may want to try all possible assignments of the inputs to low/high voltages (0/1), to see what outputs result from the different configurations. In other words, we may need to generate all possible combinations of 0 and 1 across n inputs.

Write the function **generate_inputs(n)** that takes an integer n and returns a 2D list containing all possible input combinations for n inputs. Each of the inner lists represents a single set of inputs. For example, `generate_inputs(3)` would return:

```
[ [0,0,0], [0,0,1],  
  [0,1,0], [0,1,1],  
  [1,0,0], [1,0,1],  
  [1,1,0], [1,1,1] ]
```

Because these are all the possible combinations of 0s and 1s across three values.

Hint: you'll want to use recursion to solve this problem. Think carefully about how you can make the problem smaller, and how you can use the recursive result to generate your final solution.

Note: unlike in the other recursion problems, you are allowed to use loops in this solution (in fact, you'll want to use a loop!).